BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Công nghệ kỹ thuật Điện, Điện tử

ĐỀ TÀI

TÍNH GIÁ TRỊ BIỂU THỨC HẬU TỐ BẰNG NGĂN XẾP

Mã đề DT067

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Nguyễn Thành Tân
Mã số sinh viên	N24DCDT082
Lớp	D24CQDT01-N

THÀNH PHỐ HỒ CHÍ MINH – THÁNG 10 NĂM 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Công nghệ kỹ thuật Điện, Điện tử

ĐỀ TÀI
TÍNH GIÁ TRỊ BIỂU THỨC HẬU TỐ BẰNG NGĂN XẾP

Mã đề DT067

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Nguyễn Thành Tân
Mã số sinh viên	N24DCDT082
Lớp	D24CQDT01-N
Học phần và nhóm	INT13185 Nhóm 03
Phiên bản báo cáo	Rà soát 31/10/2026; ZIP/app.py đồng bộ; C/S 17d0a9c68d8b915fa9d0327eecef36e57efa552a ; dashboard 12/12 PASS; clone sạch 5/5 unittest PASS.

THÀNH PHỐ HỒ CHÍ MINH – THÁNG 10 NĂM 2026

PHIẾU GIAO ĐỀ TÀI

Học phần INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Thông tin	Thông tin phân công và yêu cầu cá nhân DT067
Họ tên MSSV lớp	Nguyễn Thành Tân – N24DCDT082 – D24CQDT01-N (Nhóm 03)
Mã đề và tên đề	DT067 – Tính giá trị biểu thức hậu tố bằng ngăn xếp
Căn cứ giáo trình	Giáo trình; Chương 5, mục 5.3.4.2, Ví dụ 5.3; trang in 128–129, PDF 140–141
Ngày giao ngày nộp	Giao 17/10/2026; hạn nộp theo thông báo Classroom

DT067 – Tính giá trị biểu thức hậu tố bằng ngăn xếp. Input: tối đa 50 token, số nguyên 0..100, toán tử +, -, *, /, ^; số mũ 0..10; kết quả phải nằm trong Int128. Với toán tử nhị phân, lấy b (toán hạng phải) trước, lấy a (toán hạng trái) sau, tính a op b. Phép chia nguyên cắt về 0; lũy thừa kết hợp phải. Kết thúc hợp lệ khi stack còn đúng một giá trị. Hai ví dụ: 5 6 2 + * 12 4 / - = 37; 2 3 2 ^ ^ = 512. Năm ca bắt buộc: 8 3 - = 5; 3 8 - = -5; 7 2 / = 3; 1 + báo thiếu toán hạng; 4 0 / báo chia cho 0. Token số âm là mở rộng tùy chọn của chương trình.

Yêu cầu bổ sung chung: ≥12 ca kiểm độc lập; 11 ảnh thực thi/kiểm thử/clone/GitHub; bảng truy vết; lập luận tính đúng; ba quy mô hoặc cấu hình hợp lệ; URL thật, SHA mã được kiểm và permalink. Nội dung thuật toán và miền dữ liệu vẫn theo phiếu cá nhân.

Sinh viên thực hiện	Giảng viên hướng dẫn
(Ký và ghi rõ họ tên)	(Ký và ghi rõ họ tên) .....

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Không tự điền điểm khi chưa được chấm.

Tiêu chí	Trọng số	GV1 /10	GV2 /10	Quy đổi
Bài toán và đầu vào đầu ra	1.5			
Biểu diễn dữ liệu và phần tự cài	2			
Thuật toán và truy vết	2.5			
Kiểm thử và độ phức tạp	2			
Báo cáo minh chứng GitHub	1			
Bảo vệ và giải thích	1			

Quy đổi từng tiêu chí = điểm /10 × trọng số. Hai người chấm lấy trung bình điểm; một người chấm ghi rõ. Tổng điểm ...../10. Nhận xét .....

Người chấm 1 (ký, họ tên) ..... Người chấm 2 .....

CÁCH ĐỐI CHIẾU RUBRIC

Ba trạng thái Đầy đủ / Một phần / Chưa có dưới đây giúp đối chiếu nội dung và bằng chứng. Đây không phải ba mức điểm thay rubric hiện hành. Sinh viên điền vị trí bằng chứng; giảng viên xác định điểm sau khi kiểm báo cáo, mã, chạy lại và bảo vệ. Giữ cột điểm trống trước khi chấm.

Mọi tiêu chí áp dụng theo phiếu cá nhân: đề dùng biến/mảng không phải bổ sung cây, đồ thị hoặc lớp; chỉ làm nhiều thuật toán/đối chứng/cấu trúc tự cài khi phiếu yêu cầu.

C1 Đặc tả bài toán, đầu vào và đầu ra - trọng số 1.5

Đầy đủ: Đúng phiếu; miền, chỉ số và output rõ; hai ví dụ kiểm được; rỗng/trùng/hòa/vô nghiệm/input sai được nêu khi có áp dụng. Một phần: Nêu đúng bài toán chính nhưng còn thiếu hoặc chưa nhất quán một số miền, quy ước hay ví dụ. Chưa có: Chưa có đặc tả hoặc chưa có ví dụ đủ dữ liệu để đối chiếu bài toán được giao.

C2 Biểu diễn dữ liệu và phần tự cài - trọng số 2

Đầy đủ: Biểu diễn phù hợp; thao tác bắt buộc tự cài đủ; chỉ số/liên kết/trạng thái và bất biến đúng; công cụ có sẵn dùng trong phạm vi phiếu. Một phần: Biểu diễn chính dùng được nhưng còn thiếu thao tác, kiểm tra, bất biến hoặc giải thích phần tự cài. Chưa có: Chưa có biểu diễn hoạt động được hoặc chưa xác định được phần tự cài cần kiểm.

C3 Tính đúng của thuật toán và lần vết - trọng số 2.5

Đầy đủ: Thuật toán đáp ứng phiếu; lần vết khớp mã và input; giải thích được dừng, bất biến/truy hồi; kiểm nghiệm hợp lệ, tối ưu hoặc đầy đủ đúng yêu cầu. Một phần: Có thuật toán và lần vết cơ bản nhưng còn thiếu nhánh, trạng thái hoặc mắt xích lập luận; nêu rõ phần chưa kiểm được. Chưa có: Chưa có mã/lần vết đủ đối chiếu hoặc chưa có lập luận gắn thuật toán với kết quả yêu cầu.

C4 Kiểm thử, đối chiếu và độ phức tạp - trọng số 2

Đầy đủ: Đủ ca độc lập, ví dụ và ca phiếu; expected có cơ sở; actual/kết luận chạy lại được; đếm/đo và thời gian/bộ nhớ đúng bản cài, tách các pha cần thiết. Một phần: Có kiểm thử và phân tích cơ bản nhưng còn thiếu nhóm ca, cách xác định kỳ vọng, quy ước bộ đếm hoặc một phần khảo sát. Chưa có: Chưa có bảng kiểm thử đối chiếu được hoặc chưa có phân tích chi phí gắn với mã.

C5 Báo cáo, minh chứng và GitHub - trọng số 1

Đầy đủ: Đúng khung và thông tin thật; hình/bảng có dẫn giải; đủ 11 ảnh đọc được; repo/README/lệnh chạy truy cập được; phiên bản báo cáo, mã, ảnh và hồ sơ nộp đối chiếu được. Một phần: Có báo cáo và mã nhưng còn thiếu hoặc chưa rõ một số mục, ảnh, chú thích, nguồn, lệnh hay bằng chứng phiên bản. Chưa có: Chưa có báo cáo hoặc chưa có bộ minh chứng đủ để xác minh sản phẩm và phiên bản.

C6 Bảo vệ và giải thích sản phẩm - trọng số 1

Đầy đủ: Giải thích rõ bài toán, hàm, trạng thái và chi phí; dự đoán, chạy và phân tích input mới tại đúng phiên bản; nêu giới hạn sản phẩm. Một phần: Giải thích được ý tưởng/kết quả cơ bản nhưng cần gợi ý khi lần vết, giải thích mã hoặc xử lý input mới. Chưa có: Chưa có bằng chứng bảo vệ hoặc chưa giải thích được bài toán, mã và kết quả của mình.

Mỗi tiêu chí chấm 0–10. Hai người chấm: lấy trung bình tiêu chí; một người: dùng điểm người đó và ghi rõ. Điểm quy đổi = điểm tiêu chí × trọng số / 10. Cộng sáu dòng rồi làm tròn một chữ số thập phân; không làm tròn từng dòng trước khi cộng. Không tự đặt trần điểm toàn bài hoặc công thức phạt mới; ghi rõ bằng chứng còn thiếu và chấm phần kiểm được.

LỜI CAM ĐOAN

Em xin cam đoan báo cáo đồ án "Tính giá trị biểu thức hậu tố bằng ngăn xếp" được thực hiện theo phạm vi đề tài được giao, dựa trên quá trình tìm hiểu, xây dựng và kiểm thử của em. Các nội dung tham khảo từ giáo trình, tài liệu và nguồn khác đã được ghi nhận trong phần Tài liệu tham khảo.

Trong quá trình thực hiện, em có sử dụng ChatGPT/Codex để hỗ trợ tham khảo ý tưởng, rà soát lỗi và hoàn thiện giao diện, tài liệu. Em đã rà soát và chịu trách nhiệm về nội dung báo cáo, mã nguồn, kết quả kiểm thử và minh chứng được nộp. Em chỉ xác nhận là phần tự thực hiện đối với những nội dung em đã trực tiếp thực hiện, hiểu và có thể giải thích.

Em xin cam đoan các thông tin nêu trên là trung thực và xin chịu trách nhiệm về lời cam đoan này.

Thành phố Hồ Chí Minh, ngày …… tháng …… năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Nguyễn Thành Tân

TÓM TẮT

Ứng dụng Streamlit đánh giá biểu thức hậu tố DT067 bằng ngăn xếp LIFO, hiển thị lần vết từng token và tải bảng trace. Mã/ZIP đồng bộ tại commit C/S 17d0a9c68d8b915fa9d0327eecef36e57efa552a; SHA-256 app.py D37E251DE761BC9C61DA536AF5B872873BBAB878CFB592F2EA17F872FE4CEB35. Clone sạch: dashboard 12/12 PASS, unittest 5/5 phương thức PASS (12 subTest), Python 3.14.7, Streamlit 1.63.0, Pandas 3.0.5. Chia nguyên cắt về 0; validator xử lý token, miền số, thiếu/dư toán hạng, chia 0, số mũ và tràn Int128. Ảnh A01–A11 đã chèn. Cần sinh viên ký cam đoan và xác nhận khoa/ngành trước khi nộp. Repo đã công khai ngày 31/10/2026; PDF đã xuất và rà soát.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI i
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG ii
CÁCH ĐỐI CHIẾU RUBRIC iii
LỜI CAM ĐOAN iv
TÓM TẮT v
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 1
1.1. Bài toán và lý do thực hiện 1
1.2. Mục tiêu và sản phẩm 1
1.3. Phạm vi, đầu vào và đầu ra 1
1.4. Cách thực hiện và cấu trúc báo cáo 2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 3
2.1. Khái niệm và mô hình dữ liệu 3
2.2. Cấu trúc dữ liệu sử dụng 3
2.3. Nguyên lý thuật toán 4
2.4. Cơ sở đánh giá thời gian và bộ nhớ 4
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT 5
3.1. Phân tích yêu cầu chương trình 5
3.2. Thiết kế các hàm và dữ liệu 5
3.3. Thuật toán và mã nguồn chính 6
3.4. Lần vết ví dụ nhỏ 7
3.5. Giải thích tính đúng và điều kiện dừng 8
3.6. Phân tích độ phức tạp 8
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT 10
4.1. Môi trường và kế hoạch kiểm thử 10
4.2. Bảng kết quả ít nhất 12 ca 10
4.3. Hai ví dụ trên phiếu 14
4.4. Hai ca sinh viên tự thiết kế 16
4.5. Ba ca biên phù hợp với đề tài 18
4.6. Kiểm tra input không hợp lệ - LOI01 20
4.7. Kết quả toàn bộ kiểm thử 21
4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ 22
4.9. GitHub và chạy lại từ bản clone sạch 24
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 27
5.1. Mức hoàn thành so với phiếu 27
5.2. Giới hạn và bài học 27

5.3. Hướng phát triển phù hợp 27
TÀI LIỆU THAM KHẢO 29
PHỤ LỤC 1
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy 1
PHỤ LỤC B. Danh mục bằng chứng 2
PHỤ LỤC C. Tự kiểm trước khi nộp 4
PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM 5

DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Ký hiệu hoặc chữ viết tắt	Ý nghĩa trong đề tài
RPN / hậu tố	Toán tử đặt sau toán hạng; xử lý từ trái sang phải.
LIFO / stack	Phần tử đẩy sau được lấy ra trước; b phải được pop trước a trái.

DANH SÁCH BẢNG

Bảng 1.1 Đặc tả dữ liệu vào 1
Bảng 1.2 Đặc tả kết quả đầu ra 2
Bảng 1.3 Đối chiếu yêu cầu và sản phẩm 2
Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì 3
Bảng 3.1 Tổ chức các hàm và tệp 5
Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm 6
Bảng 3.3 Truy vết ca VD01 7
Bảng 3.4 Chi phí của từng pha trong bản cài 8
Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu 10
Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập 11
Bảng 4.3 Actual kết luận và vị trí minh chứng 12
Bảng 4.4 Khảo sát Q1 Q2 Q3 22
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại 24
Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu 27
Bảng PL.1 Bốn tệp cần nộp 5

DANH SÁCH HÌNH

Hình 4.1 VD01 - kết quả 37, stack cuối [37], hoàn tất 9/9 token..... 14
Hình 4.2 VD02 - kết quả 512, lũy thừa kết hợp phải..... 15
Hình 4.3 TK01 - 8 − 3 = 5..... 16
Hình 4.4 TK02 - 3 − 8 = −5..... 17
Hình 4.5 BIEN01 - 7 / 2 = 3, chia nguyên cắt về 0..... 18
Hình 4.6 BIEN02 - 0 − 7 rồi chia 2 = −3..... 19
Hình 4.7 BIEN03 - 100 ^ 0 = 1..... 20
Hình 4.8 LOI01 - ứng dụng báo lỗi chia cho 0..... 21
Hình 4.9 Bộ kiểm thử DT067 - 12/12 PASS, 0 FAIL..... 22
Hình 4.10 A10 - Clone sạch tại SHA 17d0a9c68d8b915fa9d0327eecef36e57efa552a; unittest 5/5 và dashboard 12/12 PASS.
Hình 4.11 A11 - Repo GitHub CTDL_N24DCDT082 ở commit 17d0a9c68d8b915fa9d0327eecef36e57efa552a; ảnh chụp ngày 31/10/2026 ghi nhận trạng thái riêng tư trước khi repo được chuyển công khai.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Đề tài tính giá trị biểu thức hậu tố bằng stack. Số được push; toán tử pop b trước, a sau, tính a op b rồi push. Thứ tự quan trọng với trừ, chia, lũy thừa.

1.2. Mục tiêu và sản phẩm

Mục tiêu là đánh giá hậu tố đúng thứ tự toán hạng, ghi trace theo token, trả một giá trị duy nhất hoặc thông báo lỗi cụ thể, đồng thời có bộ kiểm thử độc lập. Sản phẩm gồm app.py, test_postfix.py, requirements.txt, README.md, log kiểm thử và báo cáo theo mẫu. Tiện ích nhập trung tố chỉ chuyển biểu thức hợp lệ sang hậu tố rồi gọi cùng lõi stack.

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Theo phiếu: tối đa 50 token; số 0..100; +, -, *, /, ^; mũ 0..10; kết quả Int128. Token âm là phần mở rộng tùy chọn.

1.3.2. Kết quả và trường hợp đặc biệt

Kết quả hợp lệ là một số nguyên trong miền Int128 và chỉ được trả khi stack cuối có đúng một phần tử. Token sai, ngoài miền, thiếu/dư toán hạng, chia 0, số mũ âm/ngoài miền và tràn đều bị báo lỗi. Hòa hoặc vô nghiệm không áp dụng vì phép tính xác định với một biểu thức hợp lệ.

Bảng 1.1 Đặc tả dữ liệu vào

Tên dữ liệu	Kiểu và kích thước	Miền và điều kiện	Cách nhập hoặc khai báo
Biểu thức	Chuỗi token	1..50 token, cách nhau bằng khoảng trắng ở chế độ hậu tố	Ô nhập sidebar; có thể thay mẫu
Toán hạng	Số nguyên	0..100 theo phiếu; token âm -100..-1 là phần mở rộng	Một token số nguyên
Toán tử	Ký tự	+, -, *, /, ^; số mũ 0..10; trung gian kiểm Int128	Một token toán tử nhị phân

Bảng 1.2 Đặc tả kết quả đầu ra

Kết quả	Kiểu hoặc cách in	Ý nghĩa và điều kiện đúng	Hòa vô nghiệm hoặc lỗi
Giá trị	Số nguyên	Stack cuối có đúng một số trong Int128	-
Trace / lỗi	Bảng bước hoặc thông báo	Mỗi token có thao tác, a, b và stack sau bước	Token sai, thiếu/dư toán hạng, chia 0, mũ sai, tràn; hòa/vô nghiệm không áp dụng

Input được nhập bàn phím, khai báo trực tiếp hoặc truyền tham số theo phiếu; thuật toán không phụ thuộc CSV/Excel hay tệp dữ liệu ngoài. Nếu trường hợp hòa/vô nghiệm/rỗng không áp dụng, ghi Không áp dụng và nêu lý do.

1.4. Cách thực hiện và cấu trúc báo cáo

Quy trình thực hiện gồm đối chiếu phiếu DT067, lần vết tay, cài lõi stack, chạy test và ghi nhận kết quả. Chương 1 đặc tả dữ liệu; Chương 2 giải thích stack và nguyên lý; Chương 3 trình bày thiết kế, trace và lập luận; Chương 4 đối chiếu kiểm thử và khảo sát; Chương 5 tổng kết kết quả, giới hạn và việc còn chờ.

Bảng 1.3 Đối chiếu yêu cầu và sản phẩm

Yêu cầu trên phiếu	Sản phẩm/điều kiện kiểm	Mục sẽ trình bày
Đánh giá hậu tố bằng stack	Push số; pop b trước, a sau; tính a op b	Chương 2–3; Bảng 3.3
Chia nguyên và lũy thừa	Cắt chia về 0; ^ kết hợp phải; kiểm miền Int128	§2.3; §3.3; Bảng 4.2
Lỗi và ca kiểm	Cuối còn một giá trị; có 12 ca độc lập và trace	§3.4–3.5; Chương 4

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Hậu tố đặt toán tử sau các toán hạng. Khi đọc đến một toán tử nhị phân, hai giá trị gần nhất ở đỉnh stack là hai toán hạng của phép tính. Theo giáo trình, lấy b ở bên phải trước, rồi lấy a ở bên trái và tính a op b; thứ tự này đặc biệt quan trọng với phép trừ, chia và lũy thừa.

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Stack lưu các số đã rút gọn của phần biểu thức đã đọc. Danh sách Python dùng append để đẩy và pop để lấy đỉnh; cách LIFO phù hợp vì toán hạng gần toán tử nhất được lấy trước.

2.2.2. Quy ước và điều kiện hợp lệ

Stack ban đầu là []; đỉnh nằm ở cuối danh sách. Mỗi số làm kích thước tăng một; mỗi toán tử hợp lệ làm kích thước giảm một sau hai lần pop và tăng lại một sau push kết quả. Không có khóa trùng, cạnh hay đồ thị trong bài toán này.

Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì

Ký hiệu / cấu trúc	Ý nghĩa	Miền / bất biến
stack (list of integers)	Các giá trị của hậu tố đã rút gọn	LIFO; toán tử thay hai toán hạng bằng một kết quả
tokens / trace / snapshots	Token và trạng thái sau mỗi bước	Một bản ghi cho mỗi token; snapshot khởi đầu rỗng

Ví dụ minh họa stack

Với hậu tố VD01, sau ba token số stack là [5, 6, 2]. Token + thay 6 và 2 bằng 8; token * thay 5 và 8 bằng 40. Các token còn lại rút gọn tiếp đến [37], là kết quả duy nhất.

Cơ sở thuật toán và giới hạn áp dụng

Thuật toán áp dụng cho biểu thức hậu tố nhị phân đã token hóa. Số mũ được lấy như toán hạng phải; vì vậy 2 3 2 ^ ^ biểu diễn 2^(3^2). Input trung tố, khi được chọn, chỉ là tiện ích chuyển đổi với ngoặc và toán tử được hỗ trợ; dấu âm đơn ngôi trong trung tố chưa hỗ trợ.

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Duyệt token từ trái sang phải. Token số được kiểm miền rồi push. Toán tử yêu cầu stack có ít nhất hai số; pop b trước, pop a sau, tính a op b, kiểm chia cho 0, số mũ và Int128 rồi push kết quả. Hết token, stack phải còn đúng một số.

2.3.2. Điều kiện bắt đầu và kết thúc

Điều kiện bắt đầu là stack rỗng và token đầu tiên chưa đọc. Mỗi vòng xử lý đúng một token rồi tăng chỉ số; thuật toán dừng sau tối đa t vòng với t là số token. Bất biến là stack chứa đúng các giá trị đã rút gọn của phần hậu tố đã đọc. Kết quả chỉ được trả khi stack cuối có một phần tử.

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Lõi duyệt t token một lần nên thực hiện O(t) thao tác token và dùng O(t) bộ nhớ cho stack. Trace lưu snapshot và chuỗi stack sau mỗi token; tổng kích thước trace có thể $O(t^2)$ ở trường hợp stack tăng dần. Chi phí phép số học còn phụ thuộc độ dài số; kết quả được kiểm trong miền Int128.

Giả mã: stack rỗng; duyệt từng token, push số; với toán tử pop b rồi a, tính a op b, push kết quả; cuối yêu cầu stack có một phần tử.

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng hàm đánh giá hậu tố

Tiền điều kiện: chuỗi hậu tố có từ 1 đến 50 token, token thuộc cú pháp và miền đã nêu. Hậu điều kiện: trả một số nguyên nếu stack cuối có một phần tử; phát sinh ValueError, ZeroDivisionError hoặc OverflowError khi input/phép tính không hợp lệ.

3.1.2. Kiểm tra đầu vào

Chế độ hậu tố yêu cầu token cách nhau bằng khoảng trắng; số đầu vào từ -100 đến 100. Số âm là phần mở rộng tùy chọn.

3.2. Thiết kế các hàm và dữ liệu

Lõi tự cài là evaluate_postfix_with_trace; trunc_div thực hiện phép chia nguyên cắt về 0. infix_to_postfix và normalize_expression hỗ trợ nhập trung tố; run_test_suite tạo bảng 12 ca. Streamlit và Pandas chỉ phục vụ giao diện và hiển thị dữ liệu.

Bảng 3.1 Tổ chức các hàm và tệp

Hàm / tệp	Input	Output	Vai trò và phần tự cài
evaluate_postfix_with_trace(s)	Chuỗi hậu tố	value, trace, snapshots, tokens	Vòng lặp stack, validator và lần vết
trunc_div; infix_to_postfix	Số nguyên; biểu thức trung tố	Thương cắt về 0; chuỗi hậu tố	Chia chính xác; tiện ích nhập trung tố
run_test_suite; test_postfix.py	12 mã ca	Actual, Expected, PASS/FAIL	Dashboard và kiểm thử unittest

app.py có evaluate_postfix_with_trace, trunc_div, infix_to_postfix, normalize_expression, run_test_suite và giao diện Streamlit. Mã không có hàm apply_operator riêng. test_postfix.py kiểm 12 mã ca trong TEST_CASES cùng các phép kiểm chia chính xác, số âm tùy chọn và chuyển trung tố.

Cài đặt phần quyết định

Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm

Hàm và chữ ký	Tiền điều kiện	Hậu điều kiện	Lỗi hoặc trường hợp đặc biệt
evaluate_postfix_with_trace(s)	1..50 token hợp lệ	Trả một số và trace nếu stack cuối có một giá trị	ValueError; ZeroDivisionError; OverflowError
trunc_div(a,b)	b khác 0	Thương nguyên cắt về 0	b=0 phát sinh ZeroDivisionError
infix_to_postfix(s)	Số không âm, ngoặc, + - * / ^	Hậu tố; ^ kết hợp phải	ValueError nếu cú pháp sai

Không cần lớp, cây hay đồ thị. Stack là list; phần tử được push/pop trong hàm đánh giá. Phần tự cài cần đối chiếu là thứ tự pop b rồi a, phép tính a op b, kiểm lỗi và trạng thái trace.

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

```
        st = []
for token in tokens:
    if token la so: kiem mien; st.append(token)
    else:
        kiem len(st) >= 2
        b = st.pop()  # phai, lay truoc
        a = st.pop()  # trai, lay sau
        r = a op b; kiem chia/mu/Int128
        st.append(r)
kiem len(st) == 1; return st[0]
```

3.3.2. Đoạn mã và giải thích

Mã nguồn lõi nằm trong app.py; b được pop trước a rồi áp dụng a op b. Commit C 17d0a9c68d8b915fa9d0327eecef36e57efa552a; hàm chính: https://github.com/nguyenthanhtan06/CTDL_N24DCDT082/blob/17d0a9c68d8b915fa9d0327eecef36e57efa552a/app.py#L135; test: https://github.com/nguyenthanhtan06/CTDL_N24DCDT082/blob/17d0a9c68d8b915fa9d0327eecef36e57efa552a/test_postfix.py#L7.

3.3.3. Xử lý tình huống đặc biệt

Nhánh lỗi gồm token không hợp lệ, thiếu toán hạng, dư toán hạng ở cuối, chia cho 0, số mũ âm/ngoài 0..10, số đầu vào ngoài -100..100, quá 50 token và kết quả ngoài Int128. Các mã ca tương ứng được liệt kê tại Bảng 4.2.

3.4. Lần vết ví dụ nhỏ

VD01: 5 6 2 + * 12 4 / -. Expected 37; ảnh A01 dưới đây được chụp từ ứng dụng chạy thực tế.

Ca truy vết VD01 Input 5 6 2 + * 12 4 / - Expected 37

Bảng 3.3 Truy vết ca VD01

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
0	Khởi tạo	-	[]	Stack bắt đầu rỗng.
1	Đẩy 5	[]	[5]	Token số được push.
2	Đẩy 6	[5]	[5, 6]	Token số được push.
3	Đẩy 2	[5, 6]	[5, 6, 2]	Token số được push.
4	+; b=2, a=6	[5, 6, 2]	[5, 8]	Pop b trước, a sau; 6+2=8.
5	*; b=8, a=5	[5, 8]	[40]	Pop b trước, a sau; 5*8=40.
6	Đẩy 12	[40]	[40, 12]	Token số được push.
7	Đẩy 4	[40, 12]	[40, 12, 4]	Token số được push.
8	/; b=4, a=12	[40, 12, 4]	[40, 3]	12/4=3.
9	-; b=3, a=40	[40, 3]	[37]	40-3=37; b ở bên phải.

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
Kết thúc	Trả kết quả	[37]	37	Stack có đúng một giá trị; Expected=Actual =37.

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Bất biến: sau khi xử lý một tiền tố hợp lệ, stack chứa đúng các giá trị của những hậu tố đã rút gọn. Số được push; toán tử lấy b trước rồi a, thay hai toán hạng bằng a op b nên bất biến được duy trì.

3.5.2. Kết thúc và output

Mỗi lượt tiêu thụ một token chưa đọc; số token hữu hạn nên vòng lặp dừng sau t lượt. Theo bất biến, khi hết input, nếu stack có đúng một giá trị thì đó là giá trị của toàn biểu thức hậu tố. Nếu số phần tử khác một, input bị từ chối.

3.6. Phân tích độ phức tạp

Lõi đánh giá xử lý mỗi token một lần, O(t) thao tác token và O(t) bộ nhớ stack. Khi bật trace, snapshots sao chép stack ở mỗi bước nên phần trace có thể $O(t^2)$ về thời gian và bộ nhớ trong trường hợp xấu nhất. Không đo benchmark thời gian trong báo cáo này.

Bảng 3.4 Chi phí của từng pha trong bản cài

Pha thực hiện	Tham số quy mô	Phép toán đếm	Thời gian	Bộ nhớ phụ
Tách và kiểm token	$t \leq 50$	Đọc/tách mỗi token một lần; kiểm miền	O(t) token operations	O(t) cho token
Lõi stack	t token	Số: 1 push; toán tử: 2 pop + 1 push	O(t) token operations; chi phí số học phụ thuộc độ dài số	O(t) stack

Pha thực hiện	Tham số quy mô	Phép toán đếm	Thời gian	Bộ nhớ phụ
Kiểm lỗi và miền	Mỗi token/phép toán	Kiểm số, toán hạng, chia, mũ, Int128	O(1) kiểm tra mỗi bước ngoài chi phí phép toán	O(1) thêm
Trace/snapshot	t trạng thái	Lưu stack.copy() và chuỗi stack từng bước	$O(t^2)$ xấu nhất	$O(t^2)$ xấu nhất
Bảng/giao diện có trace	$t \leq 50$	Hiển thị trace và trạng thái	$O(t^2)$ theo kích thước trace	$O(t^2)$ theo snapshots/trace

Bảng 3.4 tách vòng lặp lõi khỏi kiểm tra đầu vào và lưu trace. Phân tích là theo số token; phép toán số nguyên có thêm chi phí phụ thuộc độ lớn toán hạng. Các con số khảo sát ở Chương 4 là phép đếm trên ba input hợp lệ, không phải kết luận benchmark thời gian.

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường kiểm thử

Môi trường kiểm từ clone sạch: Python 3.14.7, Streamlit 1.63.0, Pandas 3.0.5. Lệnh: python -m unittest -v test_postfix.py và python -c "import app; app.run_test_suite()"; kết quả lần lượt 5/5 phương thức unittest (gồm 12 subTest độc lập) và 12/12 ca dashboard PASS. SHA C/S: 17d0a9c68d8b915fa9d0327eecef36e57efa552a. Permalink và log clone-S-test-results.txt được ghi tại mục 4.9 và tệp ThongTinNop.

4.1.2. Phương pháp xác định kỳ vọng

Expected của ca số được tính tay từ thứ tự toán hạng và quy tắc chia cắt về 0. Expected của ca lỗi được xác định theo điều kiện validator; Actual lấy từ run_test_suite() và đối chiếu từng hàng. Không dùng số thực trong phép chia.

4.2. Bảng kết quả ít nhất 12 ca

Bảng 4.2 có 12 ca độc lập, Expected khớp Actual; A01–A11 đã có ảnh. Mã/ZIP ở commit C/S 17d0a9c68d8b915fa9d0327eecef36e57efa552a; clone sạch đạt 12/12 dashboard, 5/5 phương thức unittest PASS. A10 xác nhận clone; A11 xác nhận repo/commit GitHub.

Bốn ca KT09–KT12 kiểm validator riêng: thiếu toán hạng, dư toán hạng, token sai và tràn Int128. Chúng không thay thế các ví dụ hoặc năm ca bắt buộc trên phiếu.

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Bảng 4.1 ánh xạ năm ca bắt buộc với mã kiểm thử tương ứng. Hai ví dụ trong giáo trình/phiếu được báo cáo riêng tại VD01 và VD02; mỗi input, mục tiêu và phép kiểm chỉ được tính là một ca.

Ca trên phiếu	Input và yêu cầu/kỳ vọng theo phiếu	Mã ca trong bảng kiểm thử	Ảnh/log hoặc vị trí kết quả
Ca phiếu 1	8 3 - → 5; kiểm thứ tự phép trừ	TK01	Bảng 4.2; ảnh A03

Ca trên phiếu	Input và yêu cầu/kỳ vọng theo phiếu	Mã ca trong bảng kiểm thử	Ảnh/log hoặc vị trí kết quả
Ca phiếu 2	3 8 - → -5; kiểm thứ tự phép trừ	TK02	Bảng 4.2; ảnh A04
Ca phiếu 3	7 2 / → 3; chia nguyên cắt về 0	BIEN01	Bảng 4.2; ảnh A05
Ca phiếu 4	1 + → thiếu toán hạng	KT09	Bảng 4.2; test-results.txt
Ca phiếu 5	4 0 / → chia cho 0	LOI01	Bảng 4.2; ảnh A08

Các trường hợp lỗi được tính riêng vì chúng kiểm những nhánh validator khác nhau và có expected độc lập. Bộ test không đếm số assert thay cho số ca.

Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
VD01	Ví dụ phiếu	5 6 2 + * 12 4 / -	37
VD02	Ví dụ phiếu	2 3 2 ^ ^	512
TK01	Tự thiết kế	8 3 -	5
TK02	Tự thiết kế	3 8 -	-5
BIEN01	Chia dương	7 2 /	3
BIEN02	Chia âm	0 7 - 2 /	-3
BIEN03	Số mũ 0	100 0 ^	1
LOI01	Chia 0	4 0 /	Lỗi chia cho 0
KT09	Thiếu toán hạng	1 +	Lỗi thiếu toán hạng
KT10	Dư toán hạng	2 3	Lỗi dư toán hạng
KT11	Token sai	2 x +	Lỗi token
KT12	Tràn Int128	100 10 ^ 100 10 ^ *	Lỗi tràn Int128

Bảng 4.3 Actual kết luận và vị trí minh chứng

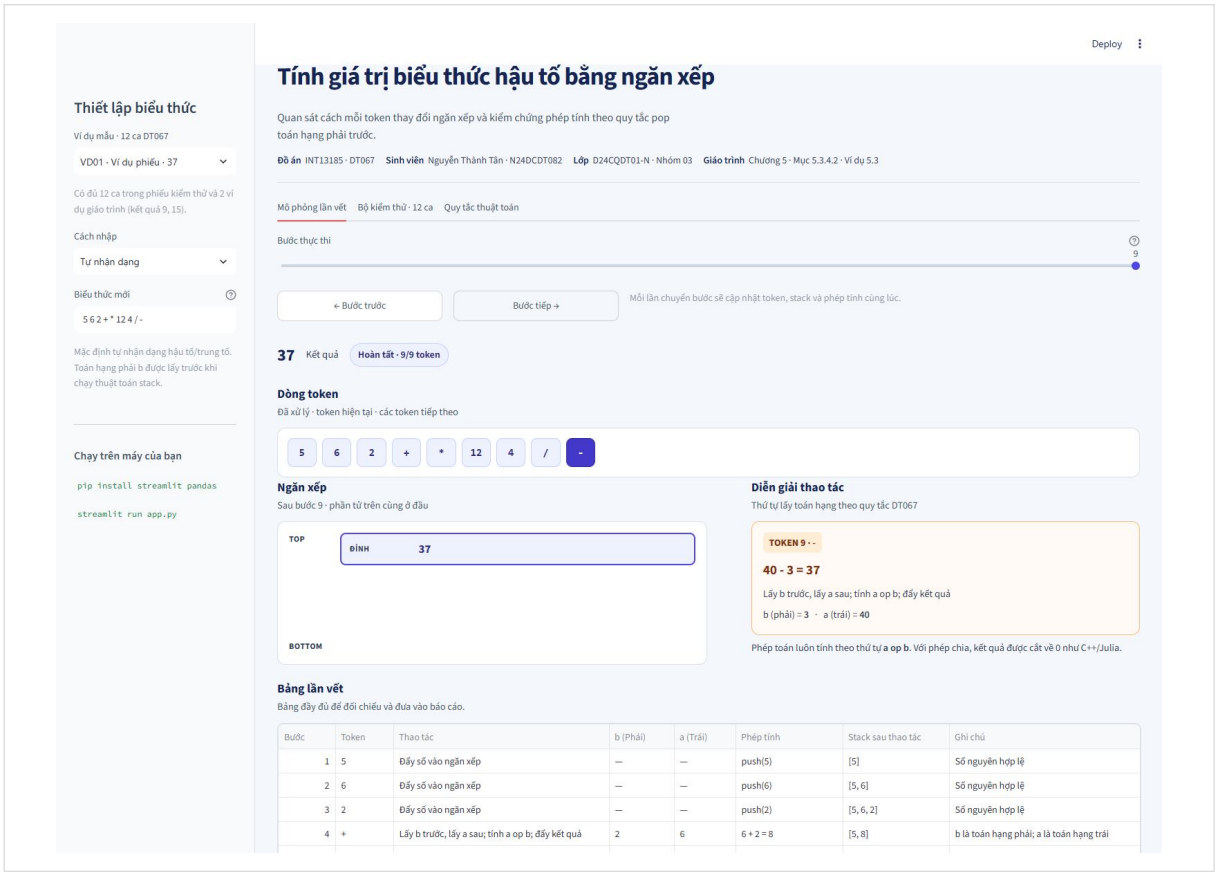
Ca	Actual	Kết luận	Ảnh log và SHA chạy
VD01	37	PASS - Expected khớp Actual	A01; Minh_chung/01_vi_du_1.png; C=17d0a9c68d8b915fa9d0327eecef36e57efa552a
VD02	512	PASS - Expected khớp Actual	A02; Minh_chung/02_vi_du_2.png; C=17d0a9c68d8b915fa9d0327eecef36e57efa552a
TK01	5	PASS - Expected khớp Actual	A03; Minh_chung/03_tu_thiet_ke_1.png; C=17d0a9c68d8b915fa9d0327eecef36e57efa552a
TK02	-5	PASS - Expected khớp Actual	A04; Minh_chung/04_tu_thiet_ke_2.png; C=17d0a9c68d8b915fa9d0327eecef36e57efa552a
BIEN01	3	PASS - Expected khớp Actual	A05; Minh_chung/05_bien_1.png; C=17d0a9c68d8b915fa9d0327eecef36e57efa552a
BIEN02	-3	PASS - Expected khớp Actual	A06; Minh_chung/06_bien_2.png; C=17d0a9c68d8b915fa9d0327eecef36e57efa552a
BIEN03	1	PASS - Expected khớp Actual	A07; Minh_chung/07_bien_3.png; C=17d0a9c68d8b915fa9d0327eecef36e57efa552a
LOI01	Lỗi chia cho 0	PASS - Expected khớp Actual	A08; Minh_chung/08_input_khong_hop_le.png; C=17d0a9c68d8b915fa9d0327eecef36e57efa552a
KT09	Lỗi thiếu toán hạng	PASS - Expected khớp Actual	test-results.txt; clone-S-test-results.txt; C=17d0a9c68d8b915fa9d0327eecef36e57efa552a; không có ảnh riêng cho ca này

Ca	Actual	Kết luận	Ảnh log và SHA chạy
KT10	Lỗi dư toán hạng	PASS - Expected khớp Actual	test-results.txt; clone-S-test-results.txt; C=17d0a9c68d8b915fa9d0327eecef36e57efa552a; không có ảnh riêng cho ca này
KT11	Lỗi token không hợp lệ	PASS - Expected khớp Actual	test-results.txt; clone-S-test-results.txt; C=17d0a9c68d8b915fa9d0327eecef36e57efa552a; không có ảnh riêng cho ca này
KT12	Lỗi tràn Int128	PASS - Expected khớp Actual	test-results.txt; clone-S-test-results.txt; C=17d0a9c68d8b915fa9d0327eecef36e57efa552a; không có ảnh riêng cho ca này

4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 - VD01

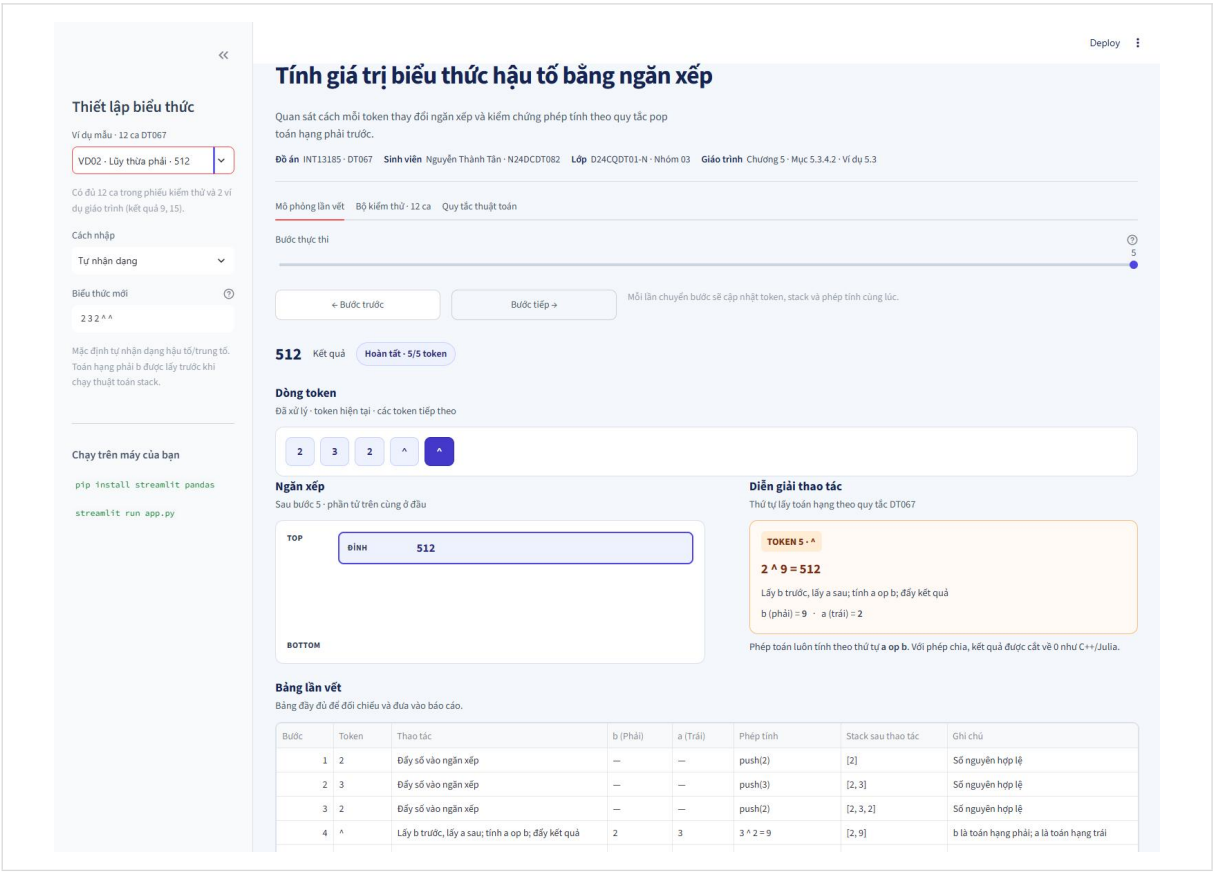
Input 5 6 2 + * 12 4 / -. Expected 37; PASS, minh chứng A01 ở dưới.



Hình 4.1 VD01 - kết quả 37, stack cuối [37], hoàn tất 9/9 token.

4.3.2. Ví dụ 2 - VD02

Input 2 3 2 ^ ^. Expected 512 vì 2^(3^2)=512; PASS, minh chứng A02 ở dưới.

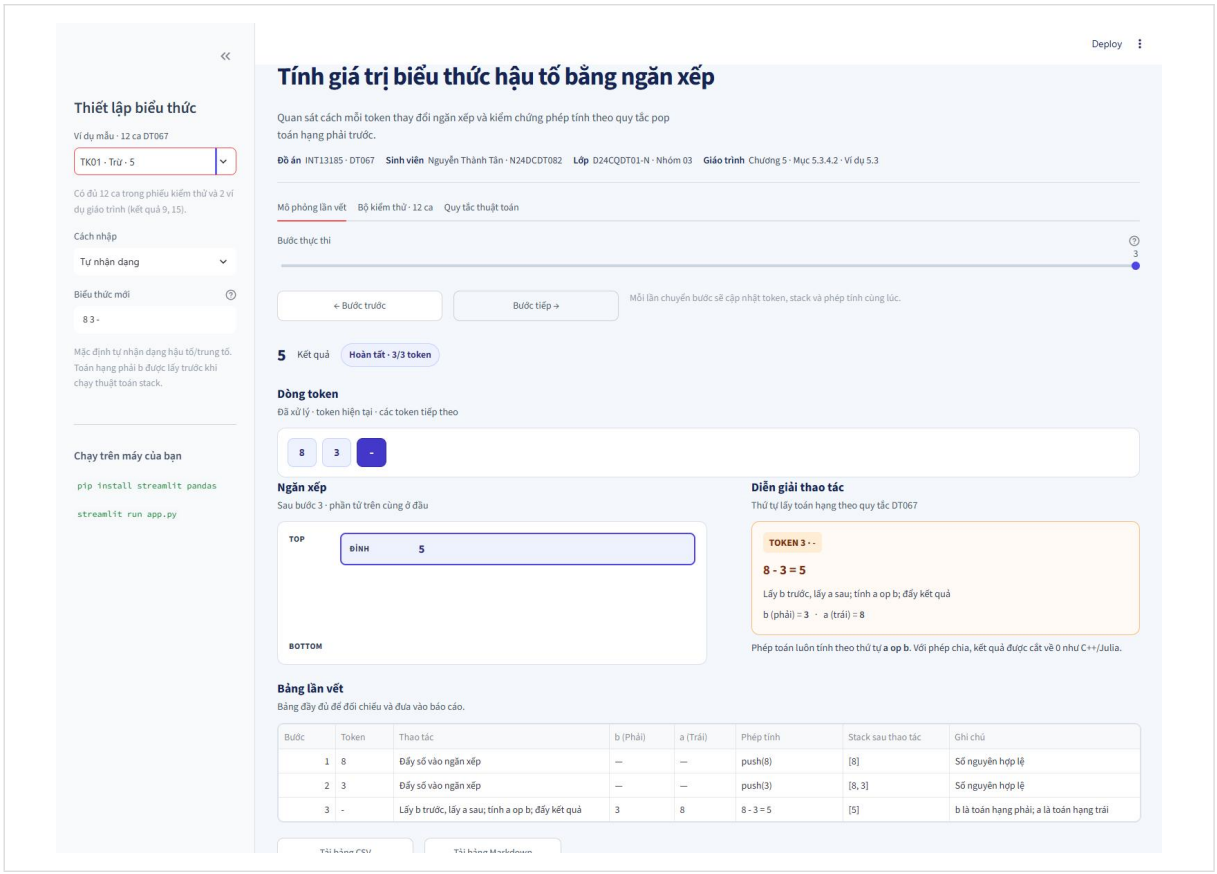


Hình 4.2 VD02 - kết quả 512, lũy thừa kết hợp phải.

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

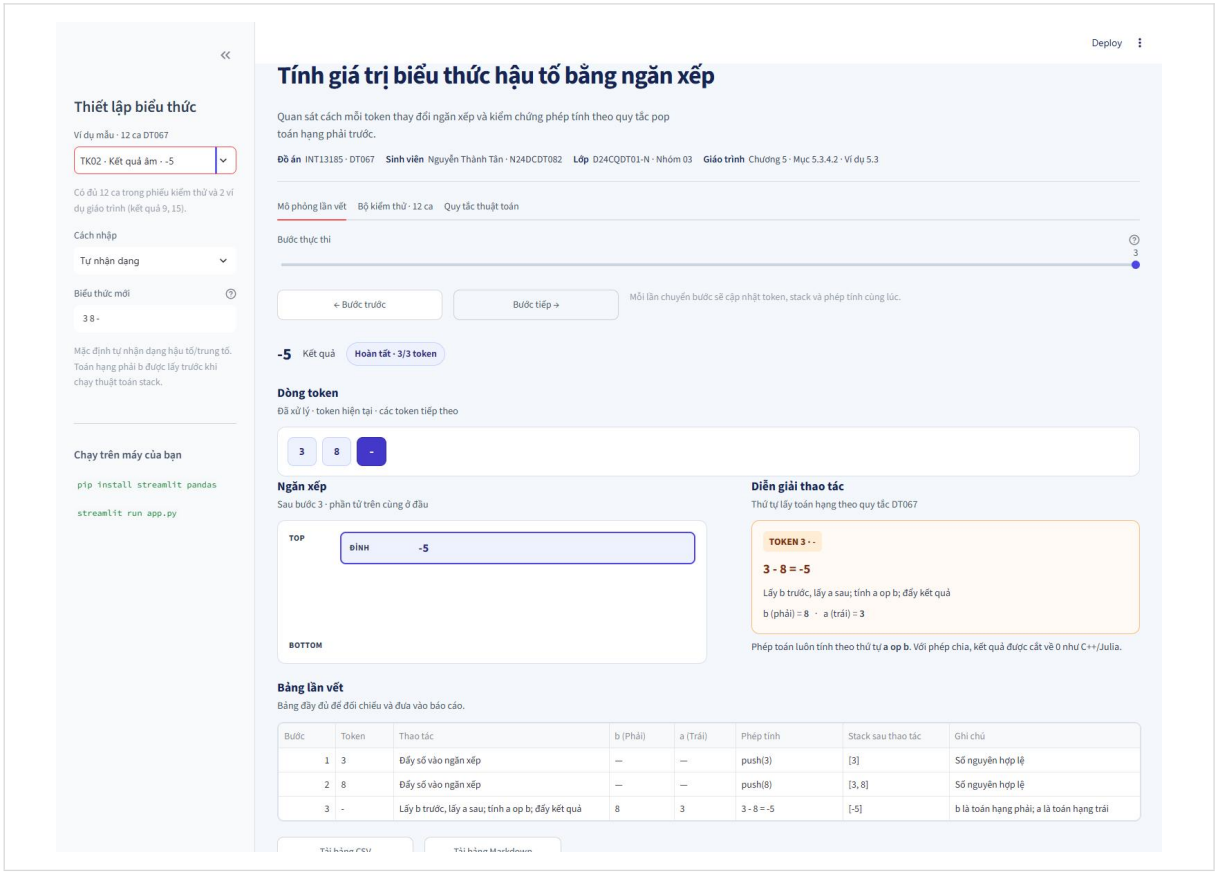
TK01: 8 3 - → 5; PASS, minh chứng A03 ở dưới.



Hình 4.3 TK01 - 8 − 3 = 5.

4.4.2. Ca TK02

TK02: 3 8 - → -5; PASS, minh chứng A04 ở dưới.



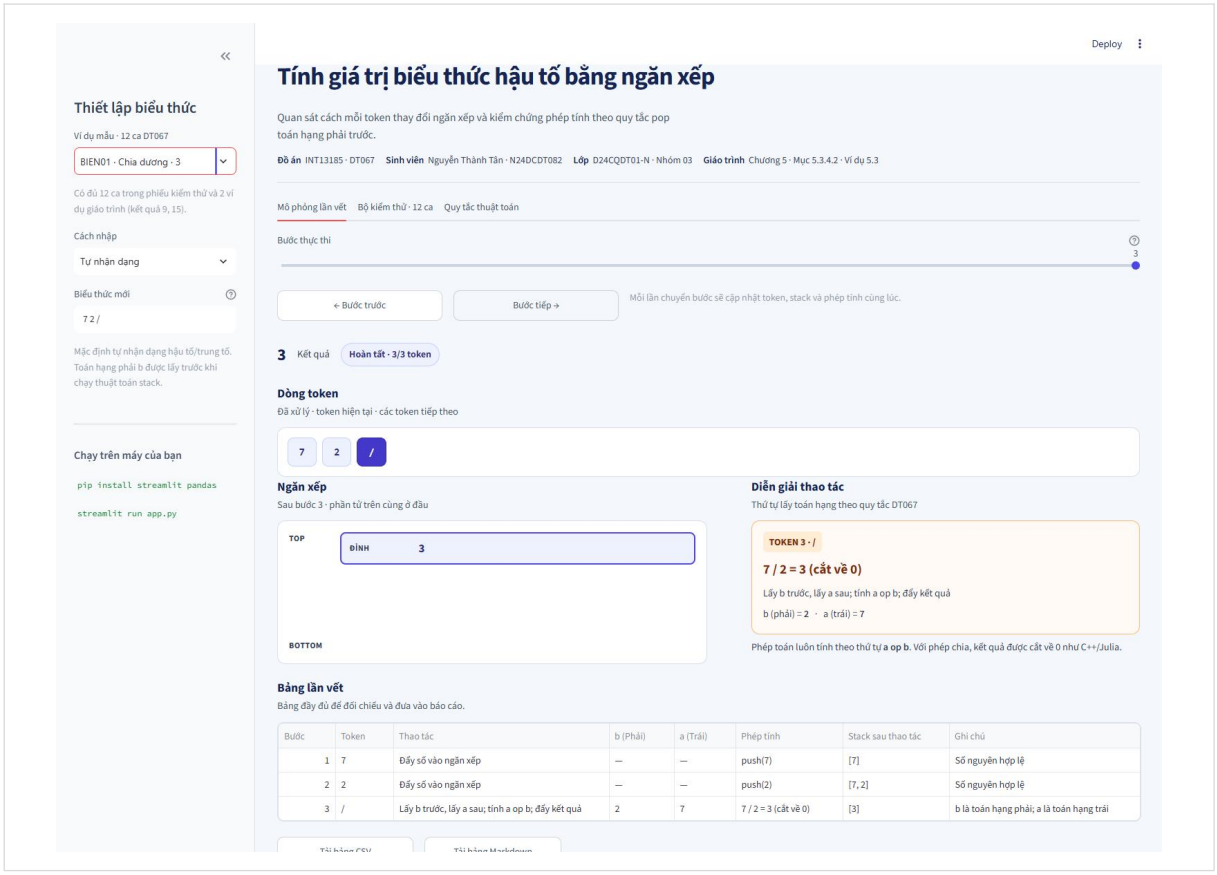
Hình 4.4 TK02 - 3 − 8 = −5.

4.5. Ba ca biên phù hợp với đề tài

4.5.1. Ca BIEN01

BIEN01: 7 2 / → 3; PASS, minh chứng A05 ở dưới.

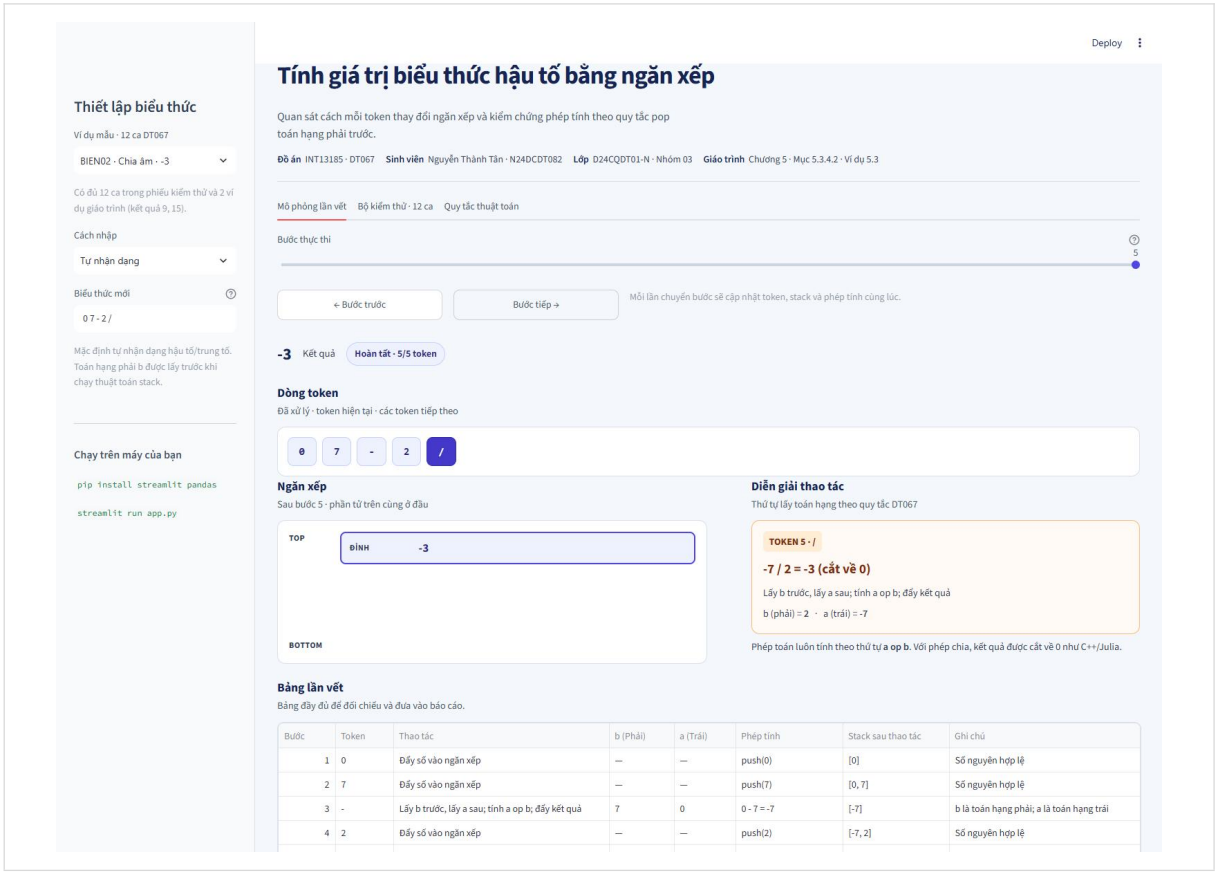
BIEN01, BIEN02 và BIEN03 đều nằm trong miền phiếu. BIEN02 tạo số âm qua phép trừ rồi kiểm quy tắc chia cắt về 0; BIEN03 kiểm số mũ 0.



Hình 4.5 BIEN01 - 7 / 2 = 3, chia nguyên cắt về 0.

4.5.2. Ca BIEN02

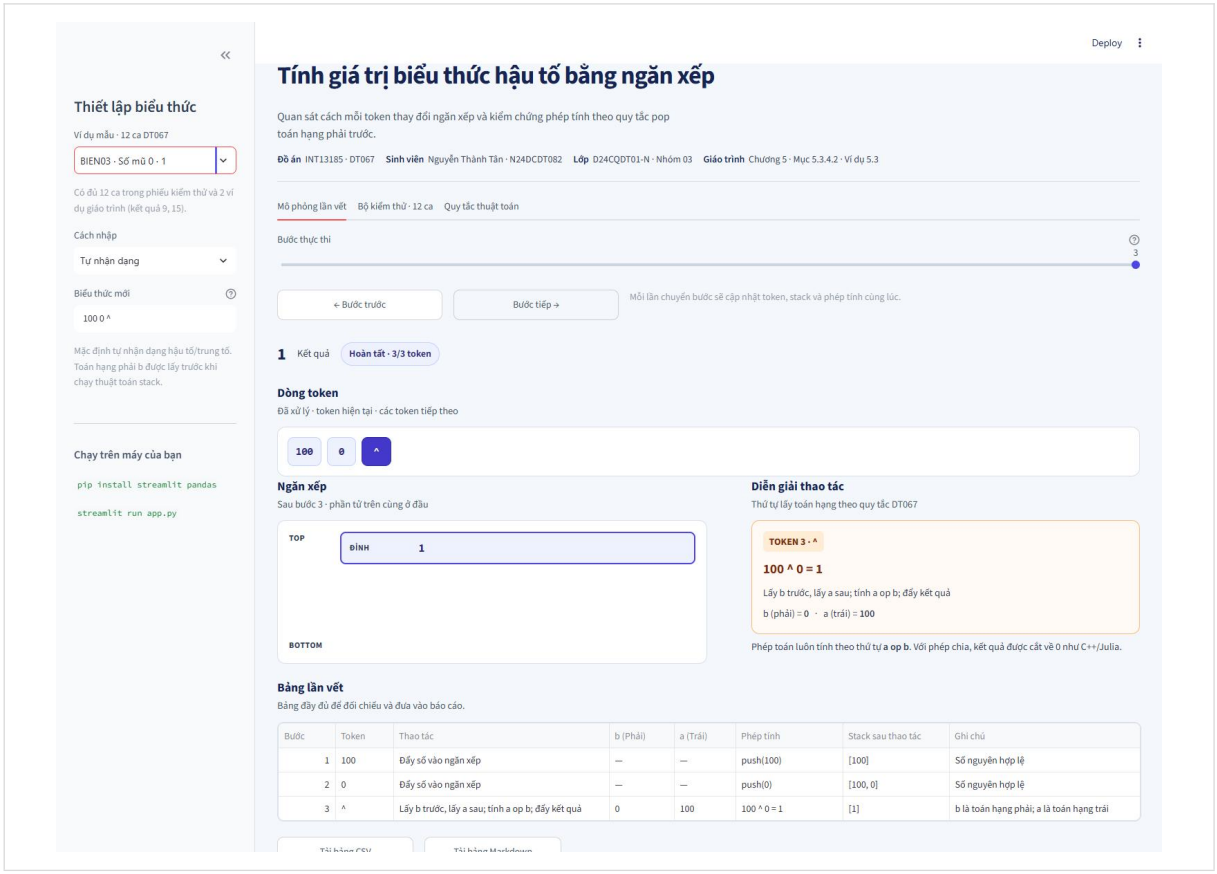
BIEN02: 0 7 - 2 / → -3; PASS, minh chứng A06 ở dưới.



Hình 4.6 BIEN02 - 0 − 7 rồi chia 2 = −3.

4.5.3. Ca BIEN03

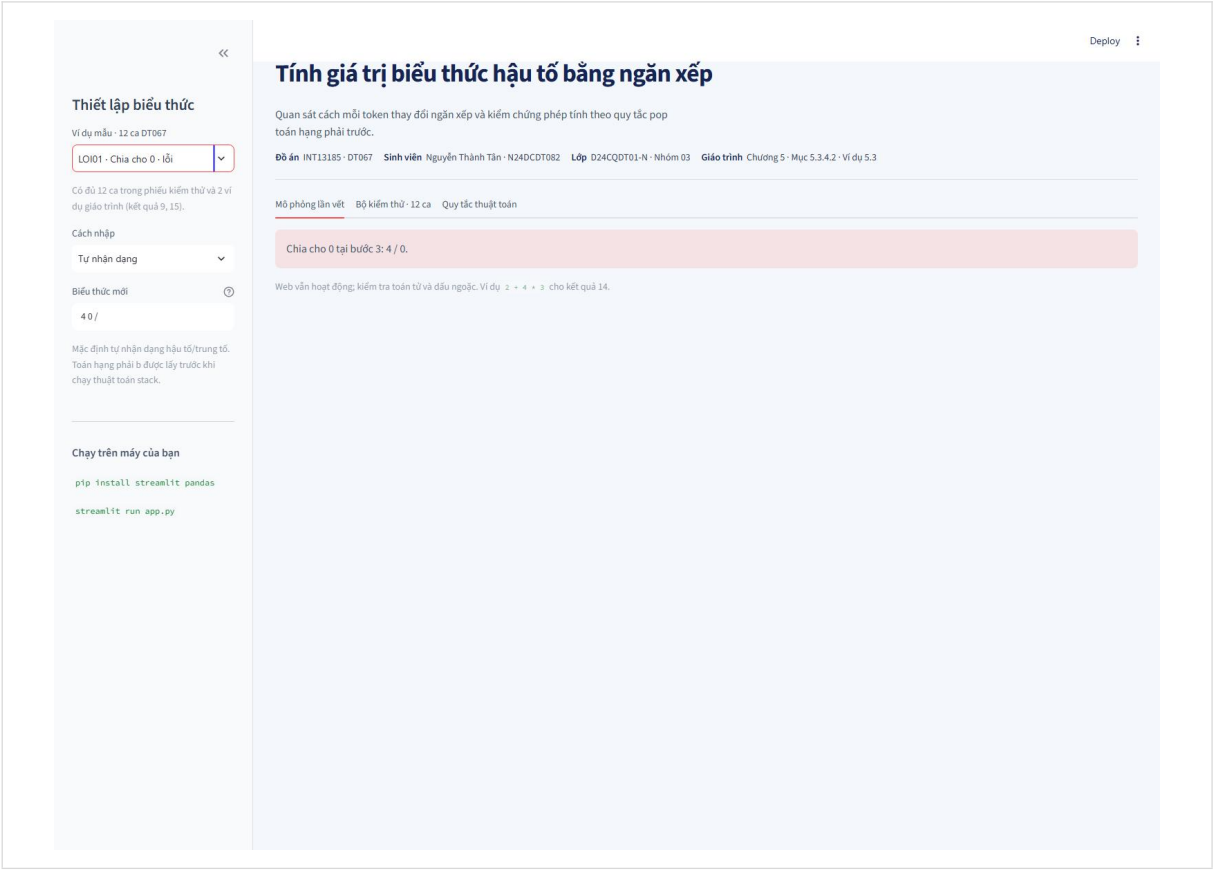
BIEN03: 100 0 ^ → 1; PASS, minh chứng A07 ở dưới.



Hình 4.7 BIEN03 - 100 ^ 0 = 1.

4.6. Kiểm tra input không hợp lệ - LOI01

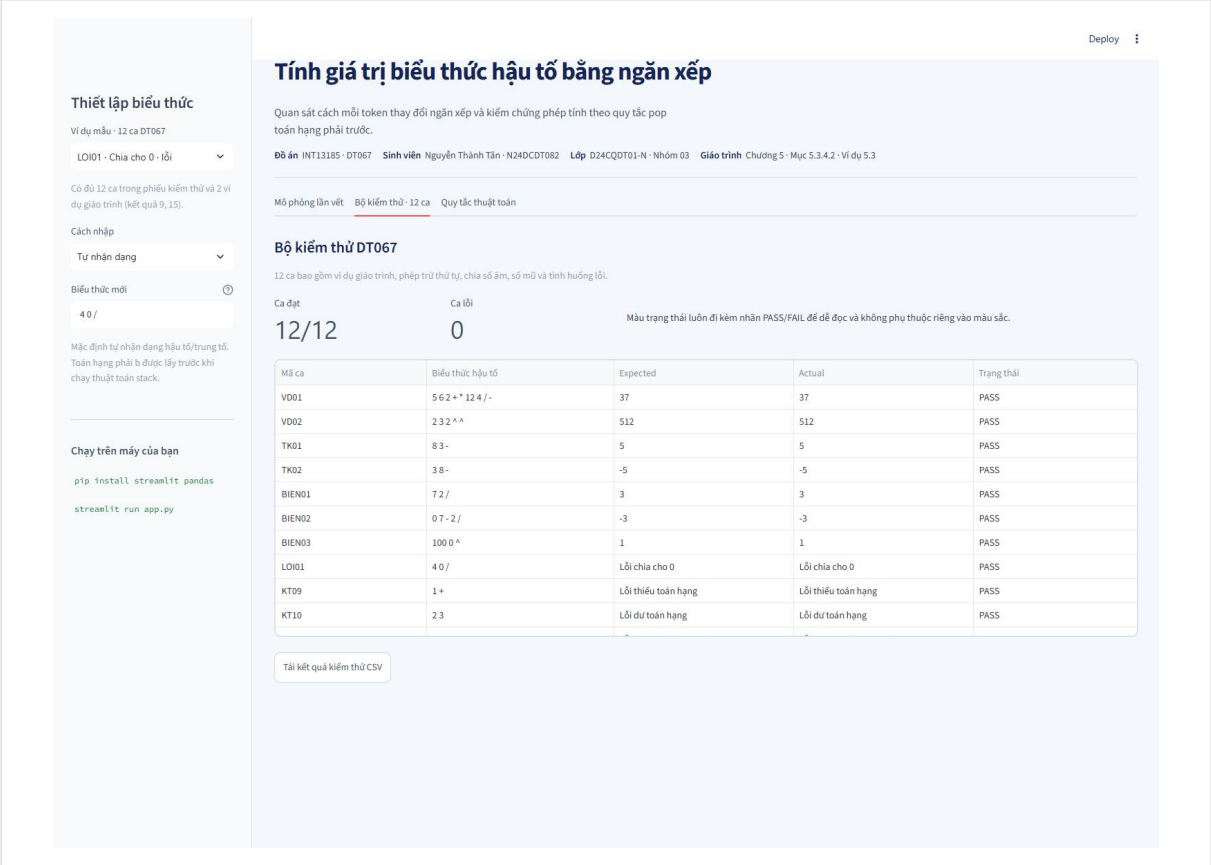
LOI01: 4 0 /; expected chia cho 0; PASS, minh chứng A08 ở trên.



Hình 4.8 LOI01 - ứng dụng báo lỗi chia cho 0.

4.7. Kết quả toàn bộ kiểm thử

ZIP mã nguồn đồng bộ với app.py tại commit C/S 17d0a9c68d8b915fa9d0327eecef36e57efa552a. Clone sạch: 12/12 dashboard PASS; unittest 5/5 phương thức PASS (12 subTest). A09 là tổng kết kiểm thử, A10 là clone sạch, A11 là repo GitHub; cả A01–A11 đã có ảnh.



Hình 4.9 Bộ kiểm thử DT067 - 12/12 PASS, 0 FAIL.

4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ

4.8.1. Dữ liệu và điều kiện khảo sát

Q1, Q2, Q3 lần lượt có 5, 15 và 41 token, đều dưới giới hạn 50. Đếm số token, số phép cộng và đỉnh stack từ input và snapshots của evaluator; không đo thời gian.

4.8.2. Kết quả và giới hạn nhận xét

Khi số token tăng từ 5 lên 15 rồi 41, số phép cộng tăng từ 2 lên 7 rồi 20; giá trị kết quả lần lượt là 6, 36, 21. Đỉnh stack là 3, 8, 21 do các input đưa nhiều số lên stack trước khi xử lý toán tử. Đây là phép đếm theo input, không phải phép đo tốc độ.

Bảng 4.4 Khảo sát Q1 Q2 Q3

Q1: 3 số, 2 cộng →6; Q2: 8 số, 7 cộng →36; Q3: 21 số, 20 cộng →21.

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q1	1 2 3 + + (5 token)	3 số, 2 phép +	Giá trị 6; stack max 3	O(t) stack; trace có 5 trạng thái
Q2	1 2 3 4 5 6 7 8 + + + + + + + (15 token)	8 số, 7 phép +	Giá trị 36; stack max 8	O(t) stack; trace có 15 trạng thái
Q3	21 token 1 rồi 20 token + (41 token)	21 số, 20 phép +	Giá trị 21; stack max 21	Trong giới hạn 50; không đo thời gian

Quy ước đếm: một token là một lần đọc; mỗi dấu + là một phép cộng; stack lớn nhất là max(len(snapshot)). Không đo thời gian nên không có bước làm nóng hoặc tổng hợp thời gian.

Cả ba input đều hợp lệ theo miền DT067 và tối đa 50 token. Số liệu chỉ mô tả lượng thao tác của ba input cụ thể, không chứng minh tốc độ tăng theo kích thước hoặc chất lượng trên input khác.

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Phiên bản có thể đối chiếu

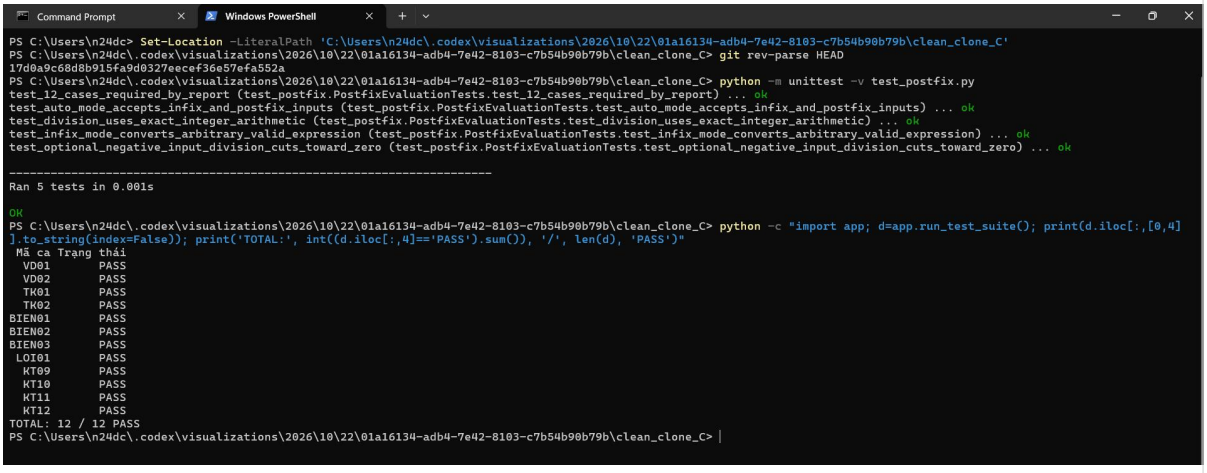
Repo công khai: https://github.com/nguyenthanhtan06/CTDL_N24DCDT082; commit nguồn C/S: 17d0a9c68d8b915fa9d0327eecef36e57efa552a; permalink: https://github.com/nguyenthanhtan06/CTDL_N24DCDT082/commit/17d0a9c68d8b915fa9d0327eecef36e57efa552a. ZIP đồng bộ với app.py. SHA-256 app.py: D37E251DE761BC9C61DA536AF5B872873BBAB878CFB592F2EA17F872FE4CEB35. Hàm evaluate_postfix_with_trace: https://github.com/nguyenthanhtan06/CTDL_N24DCDT082/blob/17d0a9c68d8b915fa9d0327eecef36e57efa552a/app.py#L135; kiểm thử: https://github.com/nguyenthanhtan06/CTDL_N24DCDT082/blob/17d0a9c68d8b915fa9d0327eecef36e57efa552a/test_postfix.py#L7. Clone sạch đạt 12/12 ca và 5/5 phương thức unittest PASS. Repo đã chuyển công khai ngày 31/10/2026; giảng viên có thể truy cập trực tiếp.

Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

Thông tin	Giá trị thật phải điền
URL repo	https://github.com/nguyenthanhtan06/CTDL_N24DCDT082 (công khai từ 31/10/2026)
SHA C	17d0a9c68d8b915fa9d0327eecef36e57efa552a; ZIP/app.py đồng bộ. SHA-256 app.py: D37E251DE761BC9C61DA536AF5B872873BBAB878CFB592F2EA17F872FE4CEB35.
SHA S	17d0a9c68d8b915fa9d0327eecef36e57efa552a; clone sạch đạt 12/12 dashboard và 5/5 unittest.
Permalink	https://github.com/nguyenthanhtan06/CTDL_N24DCDT082/blob/17d0a9c68d8b915fa9d0327eecef36e57efa552a/app.py#L135 https://github.com/nguyenthanhtan06/CTDL_N24DCDT082/blob/17d0a9c68d8b915fa9d0327eecef36e57efa552a/test_postfix.py#L7 https://github.com/nguyenthanhtan06/CTDL_N24DCDT082/commit/17d0a9c68d8b915fa9d0327eecef36e57efa552a
Môi trường	Python 3.14.7; Streamlit 1.63.0; Pandas 3.0.5
Lệnh	python -m streamlit run app.py; python -m unittest -v test_postfix.py

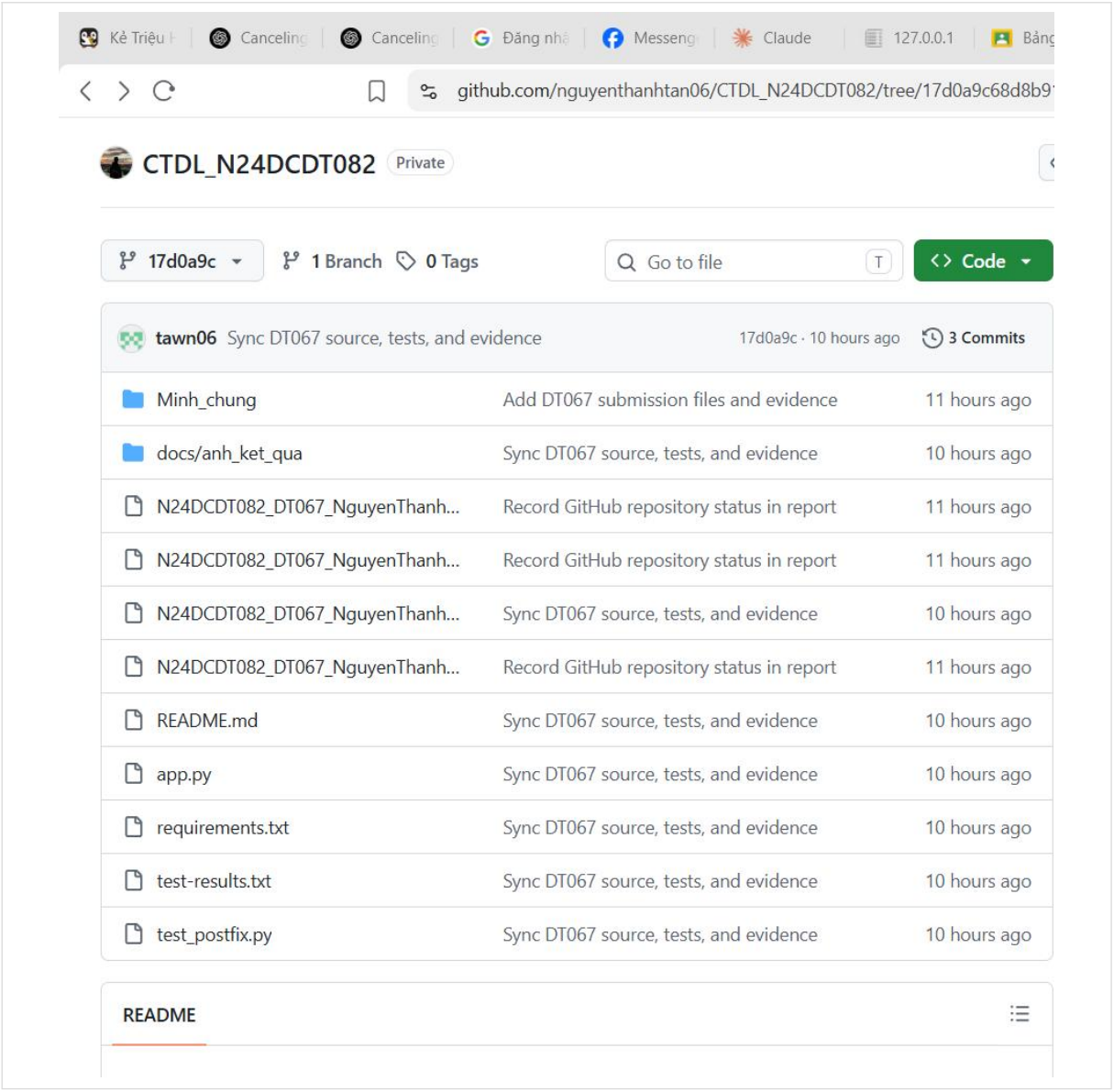
4.9.2. Clone sạch và tái hiện kết quả

Đã clone sạch repo tại commit 17d0a9c68d8b915fa9d0327eecef36e57efa552a, cài theo requirements.txt và chạy lại: dashboard 12/12 PASS, unittest 5/5 phương thức PASS. Log: clone-S-test-results.txt; ảnh A10 là Hình 4.10 và nằm trong Minh_chung/10_clone_sach_chay_lai.png.



Hình 4.10 A10 - Clone sạch tại SHA 17d0a9c68d8b915fa9d0327eecef36e57efa552a; unittest 5/5 và dashboard 12/12 PASS.

4.9.3. Bằng chứng kho GitHub thật



Hình 4.11 A11 - Repo GitHub CTDL_N24DCDT082 ở commit 17d0a9c68d8b915fa9d0327eecef36e57efa552a; ảnh chụp ngày 31/10/2026 ghi nhận trạng thái riêng tư trước khi repo được chuyển công khai.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Mã nguồn ZIP đồng bộ tại commit C/S 17d0a9c68d8b915fa9d0327eecef36e57efa552a. Clone sạch ngày 30/10/2026 đạt 12/12 dashboard và 5/5 unittest PASS (12 subTest). A01–A11 đã có ảnh. Repo đã chuyển công khai ngày 31/10/2026; sinh viên rà soát chữ ký và xác nhận khoa/ngành trước khi nộp.

5.2. Giới hạn và bài học

Miền chính là số 0..100, tối đa 50 token, số mũ 0..10 và kết quả Int128; số âm chỉ được hỗ trợ như token mở rộng. Trung tố chưa hỗ trợ dấu âm đơn ngôi. Lỗi đảo thứ tự toán hạng được phòng bằng TK01/TK02 và quy tắc pop b trước a. Khảo sát chỉ đếm thao tác, không đo thời gian.

5.3. Hướng phát triển phù hợp

Việc còn lại trước khi nộp: sinh viên rà soát, ký cam đoan và xác nhận khoa/ngành; mở rộng trung tố/dấu âm đơn ngôi chỉ khi phiếu cho phép. PDF cuối đã xuất ngày 31/10/2026, mục lục đã cập nhật và các trang đã được rà soát.

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Tiêu chí	Mức hoàn thành	Vị trí bằng chứng	Còn thiếu / ghi chú
C1 - Đặc tả	Đã điền; sinh viên rà phiếu	§1.2–1.4; Bảng 1.1–1.3; phiếu DT067	Xác nhận khoa/ngành và phiếu cá nhân
C2 - Biểu diễn	Đã trình bày	§2.2; Bảng 2.1; §3.2; app.py	Sinh viên xác nhận phần tự cài/công cụ hỗ trợ
C3 - Thuật toán/trace	Đã trình bày; A01–A02 đã chèn	§2.3; Bảng 3.3; §3.5; A01–A02; evaluate_postfix_with_trace	Giải thích trực tiếp khi bảo vệ
C4 - Test/chi phí	12/12 dashboard PASS; clone sạch 12/12; unittest 5/5 phương thức PASS	Bảng 3.4; Bảng 4.1–4.4; A03–A11; test-results.txt; clone-S-test-	12 ca đạt; khảo sát là phép đếm, không đo benchmark thời gian.

Tiêu chí	Mức hoàn thành	Vị trí bằng chứng	Còn thiếu / ghi chú
		results.txt	
C5 - Báo cáo/minh chứng	Một phần	Mẫu Word; README; ZIP; phụ lục A–D; ảnh A01–A11; URL/SHA/permalink; log clone.	Repo đã công khai ngày 31/10/2026; PDF đã xuất và rà soát; sinh viên cần xác nhận chữ ký/khoa.
C6 - Bảo vệ	Chưa xác minh	Chạy app và input mới theo Phụ lục A	Giảng viên xác minh trong buổi bảo vệ

TÀI LIỆU THAM KHẢO

Giáo trình INT1306, Chương 5, mục 5.3.4.2, trang in 128–129, PDF 140–141, Ví dụ 5.3.

Phiếu giao đề tài INT13185 Nhóm 03, DT067, 17/10/2026.

Giáo trình được dùng cho định nghĩa và quy tắc tính hậu tố; phiếu INT13185 DT067 xác định miền, phép toán, quy ước chia và các ca bắt buộc. Không dùng mã nguồn ngoài trong kết quả kiểm đã báo cáo.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

Từ PowerShell tại thư mục có app.py: py -3.14 -m venv .venv; .\.venv\Scripts\python.exe -m pip install -r requirements.txt; .\.venv\Scripts\python.exe -m streamlit run app.py. Chạy kiểm thử bằng .\.venv\Scripts\python.exe -m unittest -v test_postfix.py. Nếu lệnh py -3.14 không có, dùng python -m venv .venv.

PHỤ LỤC B. Danh mục bằng chứng

Bảng PL.2 ánh xạ ảnh và tệp. A01–A11 đã được chụp từ sản phẩm/phiên bản thật; ngày ảnh A01–A09 là 30/10/2026, A10/A11 là 31/10/2026. Repo https://github.com/nguyenthanhtan06/CTDL_N24DCDT082; commit C/S 17d0a9c68d8b915fa9d0327eecef36e57efa552a; permalink https://github.com/nguyenthanhtan06/CTDL_N24DCDT082/commit/17d0a9c68d8b915fa9d0327eecef36e57efa552a; ZIP đồng bộ.

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A01	VD01 - kết quả 37	Minh_chung/01_vi_du_1.png - chụp 30/10/2026 từ app.py hiện tại; commit C 17d0a9c68d8b915fa9d0327eecef36e57efa552a.
A02	VD02 - kết quả 512	Minh_chung/02_vi_du_2.png - chụp 30/10/2026 từ app.py hiện tại; commit C 17d0a9c68d8b915fa9d0327eecef36e57efa552a.
A03	TK01 - kết quả 5	Minh_chung/03_tu_thiet_ke_1.png - chụp 30/10/2026 từ app.py hiện tại; commit C 17d0a9c68d8b915fa9d0327eecef36e57efa552a.
A04	TK02 - kết quả -5	Minh_chung/04_tu_thiet_ke_2.png - chụp 30/10/2026 từ app.py hiện tại; commit C 17d0a9c68d8b915fa9d0327eecef36e57efa552a.
A05	BIEN01 - kết quả 3	Minh_chung/05_bien_1.png - chụp 30/10/2026 từ app.py hiện tại; commit C 17d0a9c68d8b915fa9d0327eecef36e57efa552a.
A06	BIEN02 - kết quả -3	Minh_chung/06_bien_2.png - chụp 30/10/2026 từ app.py hiện tại; commit C 17d0a9c68d8b915fa9d0327eecef36e57efa552a.
A07	BIEN03 - kết quả 1	Minh_chung/07_bien_3.png - chụp 30/10/2026 từ app.py hiện tại; commit C 17d0a9c68d8b915fa9d0327eecef36e57efa552a.
A08	LOI01 - chia cho 0	Minh_chung/08_input_khong_hop_le.png - chụp 30/10/2026 từ app.py hiện tại; commit C 17d0a9c68d8b915fa9d0327eecef36e57efa552a.
A09	TEST_ALL - 12/12 PASS	Minh_chung/09_tong_ket_kiem_thu.png - chụp 30/10/2026 từ app.py hiện tại; commit C 17d0a9c68d8b915fa9d0327eecef36e57efa552a.
A10	Clone sạch và chạy lại	Minh_chung/10_clone_sach_chay_lai.png - SHA S 17d0a9c68d8b915fa9d0327eecef36e57efa552a; unittest 5/5 và dashboard 12/12 PASS.

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A11	GitHub repo và commit	Minh_chung/11_github_repo_commit.png - repo CTDL_N24DCDT082; commit 17d0a9c68d8b915fa9d0327eecef36e57efa552a; ảnh thật.

PHỤ LỤC C. Tự kiểm trước khi nộp

Clone sạch tại commit 17d0a9c68d8b915fa9d0327eecef36e57efa552a đạt 12/12 dashboard và 5/5 phương thức unittest PASS. A01–A11 đã có ảnh trong báo cáo và thư mục Minh_chung. SHA/permalink/ZIP đối chiếu được. Còn sinh viên ký cam đoan và xác nhận khoa/ngành; repo đã công khai; PDF cuối đã xuất ngày 31/10/2026.

Rà soát hồ sơ ngày 31/10/2026; kết quả kiểm clone sạch ngày 30/10/2026 ở commit 17d0a9c68d8b915fa9d0327eecef36e57efa552a: dashboard 12/12 PASS, unittest 5/5 phương thức PASS (12 subTest). ZIP đồng bộ. A01–A11 đã chèn. Repo công khai từ ngày 31/10/2026; PDF đã xuất ngày 31/10/2026. Sinh viên cần xác nhận chữ ký và khoa/ngành.

☑ MSSV, lớp, học phần và mã đề đã được đối chiếu với thông tin có trong báo cáo; khoa cần sinh viên xác nhận.

☐ Sinh viên cần xác nhận phần tự cài, nguồn tham khảo và công cụ hỗ trợ trước khi ký.

☑ Có 12 ca độc lập; dashboard 12/12 PASS; khảo sát Q1–Q3; đủ ảnh A01–A11 đã chèn.

☑ Có trace VD01, lập luận bất biến/dừng, phân tích $O(t)/O(t^2)$ cho trace và khảo sát Q1–Q3 bằng phép đếm.

☒ Đã có URL GitHub, SHA C/S, permalink và log clone sạch tại mục 4.9; repo đã công khai ngày 31/10/2026.

☑ PDF đã xuất từ bản Word cuối ngày 31/10/2026; mục lục đã cập nhật; 46 trang đã được rà soát.

PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM

Tiền tố tệp cá nhân: N24DCDT082_DT067_NguyenThanhTan. Hạn nộp và lịch bảo vệ theo thông báo trên Classroom.

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Word	N24DCDT082_DT067_NguyenThanhTan_BaoCao.docx	Báo cáo theo mẫu; sinh viên rà nội dung/khoa, ký cam đoan; rubric để GV chấm.
PDF	N24DCDT082_DT067_NguyenThanhTan_BaoCao.pdf	Xuất từ Word cuối; cập nhật mục lục và kiểm từng trang.
Mã nguồn	N24DCDT082_DT067_NguyenThanhTan_MaNguon.zip	app.py, unittest, README, requirements, log và ảnh thật.
Thông tin	N24DCDT082_DT067_NguyenThanhTan_ThongTinNop.txt	URL GitHub, SHA C/S, permalink, lệnh và kết quả clone thật.

ZIP cá nhân gồm mã, test, README, requirements, log và ảnh thật; không đưa .git, bộ cài, môi trường ảo hoặc cache vào ZIP. Giải nén vào thư mục mới và chạy theo README trước khi nộp.

C là commit mã nguồn đã kiểm và S là commit clone sạch đã chạy lại. C = S = 17d0a9c68d8b915fa9d0327eecef36e57efa552a; SHA-256 app.py D37E251DE761BC9C61DA536AF5B872873BBAB878CFB592F2EA17F872FE4CEB35. Clone đạt 12/12 dashboard và 5/5 unittest. Ảnh A01–A11 nằm trong Minh_chung và báo cáo; log clone-S-test-results.txt.

Các thông tin để chép vào TXT

URL GitHub thật

SHA C đầy đủ SHA S đầy đủ

Permalink hàm chính và test tại C

Môi trường và phiên bản

Lệnh chạy hai ví dụ và bộ test

Kết quả clone S Số ca và phép kiểm

Khi hoàn tất các mục còn chờ, đính kèm bốn tệp Word, PDF, ZIP mã nguồn và TXT thông tin vào đúng bài tập Classroom; mở kiểm tra lại tệp rồi nộp và xác nhận trạng thái đã nộp. Không tải ảnh rời riêng nếu ảnh đã nằm trong báo cáo và ZIP.